

SGMRI

Sistema de Gestión de Mantenimiento y Reporte de Incidentes

Documentación Técnica de Ingeniería de Software e Implementación de Sistemas

Institución:	Instituto Tecnológico Beltrán
Carrera:	Técnico Superior en Análisis de Sistemas
Integrantes:	Ignacio Colatto, Emanuel Maldonado, Marcelo Siles
Año:	2026
Entorno Técnico:	Java Backend (NetBeans) / Angular Frontend / MySQL

1. Contexto General del Proyecto SGMRI

El **SGMRI (Sistema de Gestión de Mantenimiento y Reporte de Incidentes)** es una solución de software modular cliente-servidor concebida para centralizar, agilizar y transparentar los procesos de soporte de infraestructura y mantenimiento edilicio dentro del Instituto Tecnológico Beltrán.

El sistema permite a los diferentes actores de la comunidad educativa (Alumnos y Docentes) reportar anomalías físicas o técnicas en tiempo real desde dispositivos móviles o de escritorio. Asimismo, provee al personal de Mantenimiento y Administración una consola centralizada de gestión técnica, rompiendo con los canales de aviso tradicionales e informales que atentan contra la trazabilidad y la eficiencia operativa.

2. Matriz de Análisis de Riesgos (Modelo de Ingeniería)

Basado en el marco conceptual de mitigación de fallas del proyecto, se establece la siguiente matriz formal de riesgos asociados al despliegue, adopción y consistencia del SGMRI:

ID	Riesgo Identificado	Causa Raíz	Probabilidad	Impacto	Nivel Crítico
R1	Caída del sistema central	Fallo técnico en el servidor local de alojamiento o pérdida de conectividad eléctrica.	Baja	Alto	MEDIO
R2	Fuga de privilegios / Error de roles	Deficiente validación de sesiones, permitiendo a alumnos acceder a vistas de mantenimiento.	Media	Alto	ALTO
R3	Mapa de zonas desactualizado	El personal técnico ejecuta el arreglo físico pero omite reportar el cierre en el sistema.	Media	Medio	MEDIO
R4	Deficiencia en categorización	Los usuarios cargan incidentes bajo categorías técnicas incorrectas por falta de guía explícita.	Alta	Medio	MEDIO
R5	Duplicación masiva de reportes	Múltiples usuarios cargando de forma simultánea el mismo incidente del mismo aula.	Baja	Bajo	BAJO
R6	Saturación del personal operativo	Cúmulo inmanejable de reportes por incidencias menores o falta de priorización lógica.	Baja	Medio	MEDIO
R7	Resistencia cultural a la adopción	Persistencia de usuarios en reportar incidentes verbalmente en preceptoría o canales informales.	Baja	Bajo	BAJO

3. Estrategias de Mitigación Detalladas

Mitigación R2: Control Estricto de Seguridad Basado en Roles (RBAC)

Para erradicar fugas de accesos, el backend desarrollado en Java aplica filtros interceptores a nivel controlador. Toda petición proveniente del cliente Angular debe incluir el identificador del rol validado. En el frontend, se implementan Route Guards en Angular que impiden la renderización o escritura directa de rutas administrativas (como /servicios) por parte del rol Alumno.

Mitigación R3: Sincronización Automatizada del Mapa de Zonas

El sistema vincula de forma lógica la tabla de incidentes con la de zonas. En el momento en que el Personal de Mantenimiento actualiza el estado de un ticket a "En Proceso", una transacción SQL dispara de forma automática la modificación del estado_operativo de la zona a "En Mantenimiento" (Amarillo). Al pasar a "Resuelto", el aula vuelve inmediatamente a estar en estado "Operativo" (Verde).

Mitigación R4 y R5: Control de Duplicados e Interfaz Guiada

Antes de habilitar el formulario de creación de reportes, Angular obliga al usuario a visualizar el mapa de la zona. Si el aula en cuestión ya posee un indicador visual de incidente activo (color rojo o amarillo), el sistema advierte al alumno que la falla ya se encuentra en conocimiento del área técnica, mitigando la duplicación innecesaria en la base de datos.

4. Matriz de Diagnóstico FODA

FORTALEZAS

- Centralización de la gestión técnica de infraestructura en un único repositorio.
- Identificación inmediata del estado de las aulas mediante un mapa de colores dinámico.
- Arquitectura desacoplada en capas (Java/Angular) que facilita el mantenimiento futuro.

OPORTUNIDADES

- Estandarización y formalización del flujo de soporte técnico a nivel institucional.
- Generación futura de métricas operativas de rendimiento técnico (tiempos de resolución).
- Escalabilidad para incorporar el control de inventario físico del establecimiento.

DEBILIDADES

- Alta dependencia del compromiso de los usuarios para una carga ordenada de incidentes.
- Necesidad de que el personal técnico mantenga al día el cierre de los tickets.
- Curva de aprendizaje inicial para el personal operativo de mayor antigüedad.

AMENAZAS

- Caídas imprevistas de infraestructura de red local que dejen al sistema incomunicado.
- Persistencia en el uso de canales verbales informales fuera del sistema oficial.
- Falta de actualización de datos de usuarios al cambiar los ciclos lectivos.

5. Hoja de Ruta de Documentación Requerida para la Entrega Final

De acuerdo con los lineamientos académicos de Ingeniería de Software ("El profesional documenta lo justo, pero lo crítico"), para transformar el estado del proyecto de un prototipo funcional a un sistema con rigor de auditoría de software, se debe incorporar el siguiente bloque documental complementario:

A. Catálogo de Historias de Usuario Formales (Backlog)

Cada módulo funcional debe ser traducido a una historia de usuario estructurada bajo el formato estándar de la industria, acompañada por sus respectivos criterios de aceptación explícitos.

- **HU-01: Autenticación Basada en Roles**

Como Alumno / Docente / Técnico, *quiero* ingresar mis credenciales en una pantalla dedicada, *para* acceder a las funcionalidades específicas permitidas para mi rol.

Criterios de Aceptación: 1) El sistema debe verificar la contraseña contra el hash de la base de datos de manera segura. 2) Si las credenciales son erróneas, se debe notificar al usuario sin dar indicios específicos de qué campo falló por seguridad. 3) Al ingresar de forma exitosa, el sistema debe almacenar el rol de forma local para guiar la navegación.

- **HU-02: Reporte Guiado de Incidentes**

Como Alumno o Docente, *quiero* seleccionar un aula del mapa e informar un desperfecto físico, *para* que el área de mantenimiento sea notificada.

Criterios de Aceptación: 1) El campo de descripción no debe permitir texto vacío. 2) Las categorías de incidentes y las zonas deben ser selectores dinámicos extraídos de la base de datos, evitando tipeo libre erróneo.

B. Documentación y Contrato de la API REST Endpoints

Establece la interfaz técnica de comunicación (Contrato JSON) entre la capa cliente (Angular) y la capa servidora (Java en NetBeans).

Método	Ruta del Endpoint	Estructura JSON (Entrada)	Respuesta del Servidor (JSON)
POST	/api/login	{"email", "password"}	{"id_usuario", "nombre", "rol", "token"} (HTTP 200) o error 401
GET	/api/incidentes	Ninguna (Cabecera Auth)	[{"id_incidente", "titulo", "prioridad", "estado"}]
POST	/api/incidentes		

Método	Ruta del Endpoint	Estructura JSON (Entrada)	Respuesta del Servidor (JSON)
		{ "titulo", "descripcion", "id_zona", "id_cat" }	{ "status": "success", "id_incidente" } (HTTP 201)
PUT	/api/incidentes/estado	{ "id_incidente", "nuevo_estado" }	{ "status": "updated", "afectado_zona" } (HTTP 200)

C. Matriz de Casos de Prueba (Testing Plan)

Evidencia empírica formal de que el sistema fue sometido a validaciones de lógica bajo escenarios tanto positivos como negativos.

ID CP	Módulo / Escenario	Datos de Entrada	Resultado Esperado	Estado
CP-01	Login con datos inválidos	Usuario: err@test.com Clave: incorr123	Rechazo de acceso, respuesta HTTP 401 y alerta visual en formulario.	Aprobado
CP-02	Persistencia de incidente nuevo	Título: "Luz Quemada" Zona: 101, Cat: Electricidad	Escritura en tabla incidentes con ID incremental y timestamp automático.	Aprobado
CP-03	Validación de Guards de Ruta	Intento de navegación directa URL: /servicios desde cuenta Alumno	Intercepción del Guard, redirección forzada a vista /mapa.	Aprobado

D. Guía Básica de Despliegue y Configuración del Sistema

Manual enfocado en la reproducibilidad del entorno por parte del tribunal examinador para levantar el proyecto en un entorno local vacío:

- Base de Datos:** Levantar servidor MySQL local y ejecutar en consola o Workbench el archivo DDL proveído para inicializar el esquema sgMRI_db y sus registros maestros de roles y categorías.
- Servidor de Lógica:** Importar la carpeta raíz del backend dentro del IDE NetBeans. Verificar que las librerías incluyan el conector JDBC compatible (mysql-connector-java). Configurar credenciales en ConexionDB.java.

3. **Cliente Frontend:** Disponer de la plataforma Node.js instalada. Ejecutar el comando `npm install` en la raíz del proyecto Angular para descargar las dependencias, y luego utilizar `ng serve` para levantar el servidor local de desarrollo.